Chapter 7 – Data Modeling II

IERG4320/IEMS5726 Data Science in Practice | Cheat Sheet

Clustering Overview

聚类：无监督学习，把相似的数据分到同一组，没有标签

- Find groupings where data in each group is similar
- Unsupervised – no labels needed
- Uses: basket analysis, news clustering, social network communities, dimensionality reduction

Top 3 Algorithms: K-Means, Agglomerative, Spectral

K-Means Clustering

K-Means：指定K个簇，迭代移动中心点直到收敛

Algorithm

1. Initialize k centroids (random guesses)
2. Assign each point to nearest centroid
3. Recalculate centroid = mean of cluster
4. Repeat 2–3 until no change

Parameters: `k` (num clusters), `n_init` (num initializations)

Code: K-Means

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import make_blobs
from sklearn.metrics import silhouette_score
feat, true_labels = make_blobs(
  n_samples=200, centers=3,
  cluster_std=2.75, random_state=42)
scaler = StandardScaler()
scaled = scaler.fit_transform(feat)
kmeans = KMeans(init="random", n_clusters=3,
  n_init=10, max_iter=300, random_state=42)
kmeans.fit(scaled)
print("SSE:", kmeans.inertia_)
print("Centroids:", kmeans.cluster_centers_)
print("Labels:", kmeans.labels_[:5])
```

inertia_=SSE(越小越好); cluster_centers_=中心点坐标

Elbow Method – Find Best k

肘部法则：SSE随k下降，拐点处就是最佳k

```
from kneed import KneeLocator
sse = []
for k in range(1, 11):
  km = KMeans(n_clusters=k, init="random",
    n_init=10, max_iter=300, random_state=42)
  km.fit(scaled)
  sse.append(km.inertia_)
# Plot SSE vs k, find elbow
kl = KneeLocator(range(1,11), sse,
  curve="convex", direction="decreasing")
print(kl.elbow)  # e.g. 3
```

Agglomerative Clustering

层次聚类(自底向上)：每个点是一个簇，逐步合并最近的两个

Algorithm

1. Start: each point = one cluster
2. Find two closest clusters, merge them
3. Update proximity matrix
4. Repeat until all merged into one cluster

Linkage types: 计算簇间距离的方式

- **Single**: min distance between clusters
- **Complete**: max distance between clusters
- **Average**: mean distance between clusters
- **Ward**: minimize error sum of squares 最常用

Code: Agglomerative Clustering

```
from sklearn import cluster
from sklearn.neighbors import kneighbors_graph
# Ward linkage on Gaussian blobs
hc = cluster.AgglomerativeClustering(
  n_clusters=4, metric='euclidean',
  linkage='ward').fit_predict(dataset1)
# With connectivity constraints
connect = kneighbors_graph(
  dataset2, n_neighbors=5, include_self=False)
hc_conn = cluster.AgglomerativeClustering(
  n_clusters=2, metric='euclidean',
  linkage='complete',
  connectivity=connect).fit_predict(dataset2)
```

connectivity=限制只能合并相邻的点，适合非球形簇

Spectral Clustering

谱聚类：用相似度矩阵的特征值降维后再聚类，适合复杂形状

Algorithm

1. Build distance matrix → affinity matrix A
2. Compute degree matrix D, Laplacian L = D − A
3. Find eigenvalues/eigenvectors of L
4. Use top k eigenvectors as new features
5. Apply K-Means in this new space

Code: Spectral Clustering

```
from sklearn.cluster import SpectralClustering
from sklearn.datasets import make_blobs
from numpy import random
random.seed(1)
x, _ = make_blobs(n_samples=400,
  centers=4, cluster_std=1.5)
sc = SpectralClustering(n_clusters=4).fit(x)
labels = sc.labels_
# Try different cluster numbers
for i in range(2, 6):
  sc = SpectralClustering(n_clusters=i).fit(x)
  plt.scatter(x[:,0],x[:,1],s=5,c=sc.labels_)
```

默认affinity='rbf'; assign_labels='kmeans'

Clustering Evaluation Metrics

聚类没有标签，用内部指标评估簇的质量

1. Silhouette Score

轮廓系数：衡量簇内紧密+簇间分离，范围[-1,1]，越大越好

$s(i) = \frac{b(i)-a(i)}{\max(a(i),b(i))}$

a(i): avg distance to points in same cluster 越小越好

b(i): avg distance to nearest other cluster 越大越好

```
from sklearn.metrics import silhouette_score
km = KMeans(n_clusters=3, init='k-means++',
  max_iter=300, n_init=10, random_state=0)
km.fit_predict(X)
score = silhouette_score(X, km.labels_,
  metric='euclidean')
print('Silhouette:', score)  # e.g. 0.553
# Visualize with yellowbrick
from yellowbrick.cluster import SilhouetteVisualizer
vis = SilhouetteVisualizer(km,colors='yellowbrick')
vis.fit(X)
```

图中宽度=簇质量，高度=簇大小

2. Calinski-Harabasz Index

CH指数：簇间离散度/簇内离散度，越大越好

$CH = \frac{B_k/(K-1)}{W_k/(N-K)}$

B_k: between-cluster dispersion; W_k: within-cluster.

```
from sklearn import metrics
labels = kmeans.labels_
ch = metrics.calinski_harabasz_score(X, labels)
print(ch)  # e.g. 561.63, higher=better
```

3. Davies-Bouldin Index

DB指数：簇内散度与簇间距离之比，越小越好

$R_{i,j} = \frac{S_i+S_j}{M_{i,j}}, \quad DB = \frac{1}{K}\sum \max_{j\neq i} R_{i,j}$

```
from sklearn.metrics import davies_bouldin_score
db = davies_bouldin_score(X, labels)
print(db)  # e.g. 0.768, lower=better
# Compare different k values
for i in range(2, 11):
  km = KMeans(n_clusters=i, random_state=30)
  labels = km.fit_predict(X)
  print(i, davies_bouldin_score(X, labels))
```

Metric Summary:
Silhouette: $[-1, 1]$, **higher** = better 轮廓系数
Calinski-Harabasz: $[0, \infty)$, **higher** = better CH指数
Davies-Bouldin: $[0, \infty)$, **lower** = better DB指数

Reinforcement Learning (RL)

强化学习：智能体在环境中通过试错获得奖励来学习策略

Key Concepts

- **Agent**: the learner / decision maker 智能体
- **Environment**: what agent interacts with 环境
- **State** s: current situation 状态
- **Action** a: what agent can do 动作
- **Reward** r: feedback signal 奖励
- **Episode**: sequence of steps from start to end 一局游戏
- **Policy** $\pi(s) = a$: strategy for choosing actions 策略

Environment types:
Discrete (grid, board game) vs Continuous (real world)
离散环境=有限选择; 连续环境=无限可能

Exploration vs Exploitation

探索=尝试新动作; 利用=用已知最优动作

- **Exploration**: try new actions (may find better) 探索
- **Exploitation**: use known best action 利用
- **ε-greedy**: with prob ε explore randomly, else exploit
- **Boltzmann**: explore more when getting negative rewards

Q-Learning

Q-Learning: 用Q表存储每个(状态,动作)的价值

Q-Table: rows = states, cols = actions, values = expected future rewards.

Update formula:
$Q(S,A) \leftarrow Q(S,A) + \alpha\left[R + \gamma \max_{A'} Q(S',A') - Q(S,A)\right]$
α = learning rate, γ = discount factor, R = reward.
当前Q值 + 学习率 × (实际回报 - 当前估计)

Code: Q-Learning Agent (Tic-Tac-Toe)

```
class QLearningAgent:
  def __init__(self, alpha=0.1, gamma=0.9,
               epsilon=0.1):
    self.q_table = defaultdict(
      lambda: np.zeros(9))
    self.alpha = alpha     # learning rate
    self.gamma = gamma     # discount factor
    self.epsilon = epsilon # exploration rate

  def get_action(self, state, available):
    state_key = self._state_to_key(state)
    if random.random() < self.epsilon:
      return random.choice(available) # Explore
    else:
      q = self.q_table[state_key]      # Exploit
      avail_q = {m: q[m] for m in available}
      return max(avail_q, key=avail_q.get)

  def update_q_value(self, state, action,
                     reward, next_state,
                     next_avail):
    sk = self._state_to_key(state)
    nk = self._state_to_key(next_state)
    cur_q = self.q_table[sk][action]
    if next_avail:
      max_nq = max(self.q_table[nk][m]
                   for m in next_avail)
    else:
      max_nq = 0  # game over
    # Q-learning formula
    new_q = (1-self.alpha)*cur_q + \
        self.alpha*(reward+self.gamma*max_nq)
    self.q_table[sk][action] = new_q
```

RL Algorithm Categories

RL算法分类

1. Model-Free vs Model-Based:

- **Model-Free**: learn policy directly (Q-Learning, DQN, Policy Gradient) 不建环境模型
- **Model-Based**: learn environment dynamics (AlphaZero, Dyna-Q) 先学环境再规划

2. Value vs Policy vs Actor-Critic:

- **Value-Based**: learn V(s) or Q(s,a), derive policy (Q-Learning, DQN) 学价值函数
- **Policy-Based**: directly learn $\pi(a|s)$ (REINFORCE) 直接学策略
- **Actor-Critic**: hybrid of both (PPO) 结合两者

RL Evaluation

1. Win rate change during training 训练过程中胜率变化
2. Win rate vs random agent in testing 测试时对随机agent的胜率
3. Number of different moves explored (Q-Table size) Q表中探索的状态数
4. Feedback from skilled opponents 专家评价